Constructive and Destructive Training Regimes: A Framework for Interpreting SGD Stability

Thiago Mattar, Cristiano L. Castro, Frederico Coelho, Yan G. Ferreira, Antonio P. Braga

***Abstract*—Parameter updates can reinforce or cancel one another even when training loss continues to decrease. We study this distinction through an exact finite-time decomposition of displacement under stochastic gradient descent (SGD), separating gradient magnitudes from interactions across all temporal lags. Consecutive-gradient cosine provides a local directional diagnostic, while normalized displacement measures cumulative retention of traveled distance. Local expansions connect these quantities to curvature and sampling noise without identifying alignment sign with stability or objective progress. Full-batch experiments exhibit gradient reversal alongside continued loss reduction. Matched mini-batch continuations show that sampling changes both gradient estimates and the paths traversed through the objective. We then extend the diagnostic interpretation to Adam by distinguishing raw gradients, first moments, adaptive directions and actual parameter increments. Language-model measurements and checkpoint-matched schedule interventions demonstrate why coherence at these different stages must be assessed alongside loss. The resulting framework supports the formulation and evaluation of informed learning-rate interventions; it does not prescribe an alignment-maximizing schedule.**

***Index Terms*—Stochastic Gradient Descent, Adam, Gradient Alignment, Parameter Displacement, Learning-Rate Scheduling**

I. Introduction

Neural scaling laws have ushered in new paradigms for interpreting deep learning dynamics. By establishing that performance gains follow predictable behavior relative to model size, dataset scale, and compute, stochastic optimization methods have been pushed toward increasingly extended training horizons while operating under tight efficiency constraints [1]–[4]. In this regime, optimization stability becomes an important practical requirement, since unstable training may result in slow convergence and high sensitivity to hyperparameter choices. In practice, stability is often maintained through a set of heuristics, such as momentum, batch-size scaling, normalization, and learning-rate schedules [5]–[9]. Yet, despite their effectiveness, it is not fully understood why these mechanisms work, and under what conditions they fail.

Thiago Mattar, and Yan G. Ferreira are with the Graduate Program in Electrical Engineering, Universidade Federal de Minas Gerais, Belo Horizonte 31270-901, Brazil (e-mail: thiagocmattar@gmail.com; yanvgferreira@gmail.com).

Cristiano L. Castro is with the Graduate Program in Electrical Engineering and the Department of Electrical Engineering, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Minas Gerais 31270-901, Brazil (e-mail: crislcastro@gmail.com).

Frederico Coelho, and Antonio P. Braga are with the Graduate Program in Electrical Engineering and the Department of Electronic Engineering, Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, Minas Gerais 35931-008, Brazil (e-mail: fredgfc@ufmg.br; apbraga@ufmg.br).

Classical local analyses relate the permissible step size to curvature in sharp directions of the loss [10], [11]. Stochastic extensions, including the break-even point framework, also account for curvature variation induced by sampling [12]. The Edge-of-Stability literature shows that deep networks can continue to make progress while oscillating near, or beyond, the nominal sharpness threshold [13]–[15]. These results motivate a complementary diagnostic question: how do the magnitudes and directions of successive gradients combine into parameter motion along the observed training trajectory? Gradient information already computed during optimization makes this question accessible without estimating the full Hessian spectrum or gradient-noise covariance at every update.

To examine these interactions, we describe parameter motion through gradient magnitudes and directional alignment. Starting from an untrained reference, training changes the parameters through which the model expresses what it has learned. Displacement provides a geometric description of that change; it does not quantify retained knowledge or establish whether the change improves the objective. Positive and negative gradient interactions respectively reinforce and cancel components of the accumulated motion. Their effect must be evaluated at the relevant temporal horizon and distinguished from loss reduction.

We develop an exact finite-time decomposition of SGD displacement that retains all cross-step interactions. Consecutive-gradient cosine supplies a local online diagnostic, while normalized displacement measures the fraction of accumulated step length retained as net motion. Local expansions relate gradient changes to directional curvature and sampling noise. The distinction between these quantities is consequential: a trajectory can reverse successive gradient directions while continuing to reduce loss, and sampling can change both the measured alignment and the subsequent path.

The same distinction becomes essential under adaptive optimization. Adam already computes raw gradients and moment estimates [16]; its parameter updates combine temporal averaging, coordinate scaling and the learning rate. Monitoring these successive quantities can reveal changes that a single alignment statistic would obscure. We use language-model measurements and schedule interventions to illustrate their interpretation, while separating descriptive geometric responses from the development of an improved learning-rate rule.

Our contributions are:

- A displacement-based account of local and cumulative gradient interaction, with explicit connections to curvature, sampling noise and finite-step loss changes;

- Controlled full-batch and mini-batch experiments that distinguish directional reversal, displacement retention and objective progress;
- A practical extension to Adam that separates raw-gradient information, momentum, adaptive directions and actual parameter motion, supported by language-model measurements and matched schedule interventions.

The remainder of the paper is organized as follows. Section II reviews training-stability perspectives. Section III develops the displacement formulation and its diagnostics. Section IV examines full-batch GD and mini-batch sampling. Section V extends the interpretation to Adam and outlines the empirical requirements for informed scheduling. Section VI concludes.

II. Background: Stability Analysis of SGD

Training stability concerns how perturbations and loss evolve under repeated updates. Overshoot and oscillation are useful symptoms, but can also accompany sustained objective progress. Understanding these dynamics requires examining both the *local curvature* of the loss landscape and the *stochasticity* introduced by mini-batch sampling [10], [11], [13].

A. Local Curvature of the Loss Function and Stability

Training can be interpreted as a discrete-time dynamical system in which the local curvature of the loss function determines the stability of the updates [10], [11]. The curvature indicates how sensitive the loss is to parameter changes in different directions: when curvature is small, updates tend to produce smooth descent toward a minimum, whereas in directions of high curvature the same step can overshoot the minimum and induce oscillatory behavior [10], [11], [17]. For this reason, classical analyses of gradient descent relate stability to the spectrum of the Hessian of the loss function, whose eigenvalues quantify directional curvature and indicate how sensitive optimization is to the choice of learning rate [10], [11].

Under this view, let $\theta_t \in \mathbb{R}^p$ denote the parameters after update t, and let g_t denote a mini-batch gradient evaluated at θ_{t-1}. For a constant learning rate $\eta > 0$, the SGD update is

$$\theta_t = \theta_{t-1} - \eta\, g_t. \tag{1}$$

For full-batch GD, $g_t = \nabla\mathcal{L}(\theta_{t-1})$, where $\mathcal{L}$ denotes the full training objective. Defining $H_t := \nabla^2\mathcal{L}(\theta_{t-1})$, the Jacobian of the deterministic update map is

$$J_t = I - \eta\, H_t. \tag{2}$$

Consider a small perturbation e_{t-1} around θ_{t-1}. Under the local linearization of the full-batch GD update, this perturbation evolves as

$$e_t \approx J_t e_{t-1} = (I - \eta H_t) e_{t-1},$$

In a locally quadratic regime near a minimum, the Hessian is approximately constant and admits eigenpairs (λ_i, v_i). Decomposing the perturbation as $e_t = \sum_i \delta_t^{(i)} v_i$, each eigendirection evolves independently according to

$$\delta_t^{(i)} = (1 - \eta\lambda_i)\, \delta_{t-1}^{(i)}.$$

Thus, the factor $1 - \eta\lambda_i$ determines whether perturbations along direction v_i decay or grow. For a positive-definite Hessian, a classical necessary and sufficient condition for linear stability is

$$\max_i |1 - \eta\lambda_i| < 1 \;\Leftrightarrow\; \eta\,\lambda_{\max}(H_t) < 2, \tag{3}$$

which means that all perturbation components contract over time [10], [11]. In other words, if $0 < \eta\lambda_i < 1$, the perturbation decays monotonically along v_i; if $1 < \eta\lambda_i < 2$, it alternates sign while shrinking, producing oscillations around the minimum; and if $\eta\lambda_i > 2$, it alternates sign while growing, leading to divergence. In over-parameterized networks, however, the Hessian spectrum is typically highly anisotropic, with a small number of outlier directions carrying most of the sharpness, while the bulk remains near zero [18], [19]. Although this suggests monitoring $\lambda_{\max}(H_t)$ as a stability diagnostic, estimating leading Hessian eigenvalues online in large neural networks is computationally expensive, and mini-batch estimates provide only noisy, pointwise proxies of the local geometry.

B. Mini-Batch Stochasticity

A complementary line of work studies SGD through the lens of stochastic optimization, interpreting the mini-batch gradient as a noisy approximation of the full gradient computed over the entire dataset [20]–[23]. In this sense, each SGD step combines a deterministic descent component with a zero-mean random fluctuation induced by sampling, which covariance provides information about how noise is distributed across parameter-space directions and how it may affect trajectory stability. Formally, under fresh, conditionally unbiased sampling, we write the mini-batch gradient as

$$\begin{aligned} g_t &= \nabla\mathcal{L}(\theta_{t-1}) + \xi_t, \\ \mathbb{E}[\xi_t \mid \theta_{t-1}] &= 0, \\ \mathrm{Cov}(\xi_t \mid \theta_{t-1}) &\approx \frac{1}{b}\,\Sigma(\theta_{t-1}), \end{aligned} \tag{4}$$

where ξ_t is the sampling noise, b is the batch size, and $\Sigma(\theta_{t-1})$ denotes the covariance matrix of per-sample gradients around the full gradient. At the current parameters, write $h_i := \nabla\ell_i(\theta_{t-1})$, where ℓ_i is the loss for sample i, and $\mu := \frac{1}{N}\sum_{i=1}^N h_i = \nabla\mathcal{L}(\theta_{t-1})$, where N is the dataset size. The covariance matrix is

$$\Sigma(\theta_{t-1}) := \frac{1}{N}\sum_{i=1}^{N}\big(h_i - \mu\big)\big(h_i - \mu\big)^\top, \tag{5}$$

and the scalar dispersion of per-sample gradients around the full gradient is given by the trace of $\Sigma(\theta_{t-1})$

$$\mathrm{tr}(\Sigma(\theta_{t-1})) = \frac{1}{N}\sum_{i=1}^{N}\|h_i - \mu\|^2.$$

This makes clear why $\Sigma(\theta_{t-1})$ is generally not a viable online diagnostic: computing it requires access to the full gradient μ and all per-sample gradients $\{h_i\}_{i=1}^N$ at each training step, which is impractical and incurs at least a full-pass cost per step.

C. Curvature–Noise Interaction

The *break-even point* framework of Jastrzebski et al. [12] is an attempt to extend the classical curvature-based stability condition to SGD. The central idea is to analyze stability along the sharpest direction of the loss landscape while explicitly accounting for mini-batch stochasticity. More precisely, the framework proposes a one-dimensional necessary-and-sufficient condition for SGD stability at iteration t,

$$(1 - \eta\lambda_{\max}(H_t))^2 + s_t^2 \frac{\eta^2(N-b)}{b(N-1)} \;\leq\; 1, \tag{6}$$

where $\lambda_{\max}(H_t)$ is the top Hessian eigenvalue, N is the dataset size, b is the batch size, and s_t^2 denotes the variance across training examples of the curvature measured along the leading Hessian direction. Large s_t^2 means different mini-batches see substantially different local curvature, while small s_t^2 means they see similar curvature. In this sense, Eq. (6) can be read as a stochastic extension of the deterministic stability condition in Eq. (3), so that the first term captures curvature-induced amplification, while the second captures the destabilizing effect of mini-batch stochasticity.

Under this definition, the break-even point is defined as the first iteration at which the left-hand side of Eq. (6) reaches 1. Intuitively, it marks the end of an early phase during which the choice of learning rate and batch size can strongly shape the later optimization trajectory. In this sense, a central conjecture of this framework is that, prior to this point, hyperparameters largely determine the maximum curvature and stochasticity levels encountered later in training, thereby setting the regime in which the subsequent dynamics unfold.

Although the local analysis simplifies both the interpretation and the mathematical treatment of the stability problem, it is subject to well-known limitations. For instance, the Edge-of-Stability (EoS) phenomenon challenges the local-quadratic picture even in deterministic gradient descent (GD). Across architectures and tasks, full-batch GD often exhibits progressive sharpening, with the largest eigenvalue of the Hessian, $\lambda_{\max}(H_t)$, rising until it hovers near (and sometimes above) the nominal threshold $2/\eta$ (Eq. 3), while optimization continues to make progress despite short-term non-monotonicity [13]–[15]. Thus, successful training can coexist with local linear instability for extended periods, indicating that sharpness alone does not fully explain and determine trajectory-level behavior.

Together, these results clarify both the value and the limitation of curvature–noise interaction criteria. Quantities such as the leading Hessian eigenvalue $\lambda_{\max}(H_t)$ and the gradient-noise covariance $\Sigma(\theta_{t-1})$ characterize local geometry and stochasticity, but can be expensive to estimate online. Their pointwise values also leave open how successive updates interact and accumulate over a training trajectory. This motivates complementary diagnostics that describe directional reinforcement and cancellation using quantities available during optimization.

D. Practical Stability Analysis in Large-Model Pretraining

Large-model training reports approach stability through observable symptoms and interventions on the training recipe. An instructive example is PaLM: Section 5.1 reports approximately 20 loss spikes during training of the 540B model despite global gradient clipping. The authors recovered by returning to a checkpoint about 100 steps before a spike and skipping 200–500 batches. Replaying those batches from a different parameter state did not reproduce the spikes, suggesting an interaction between data and model state rather than a property of the batches alone [24].

Reports accompanying released model weights provide further operational evidence. During OPT-175B training, loss divergence was associated with failures of the dynamic loss scale and growing final-layer activation norms. Recovery involved reducing the learning rate and restarting from an earlier checkpoint; lowering the gradient-clipping threshold from 1.0 to 0.3 also improved stability early in training [25]. These observations motivate monitoring the numerical state of training alongside the loss.

OLMo 2 makes this diagnostic process explicit by measuring a rolling spike score on training loss and gradient norm, and using controlled ablations to study initialization, normalization, and optimizer settings. Its revised recipe combines global gradient clipping with changes including query–key normalization, reordered normalization, and a logit regularizer (z-loss) [26]. In Llama 3, the authors report a stable pretraining recipe with few loss spikes and no interventions for divergence; they use learning-rate warmup and a staged batch-size schedule, and emphasize FP32 gradient accumulation and reduction for numerical stability [27]. These reports document both failure recovery and recipes that avoid recurrent failures, rather than a universal instability mechanism.

Loss, gradient norm, and activation statistics are practical monitoring signals, but gradient magnitude alone does not describe the relationship between successive directions. A large gradient can reinforce or oppose its predecessor, and a sequence of individually bounded steps can still exhibit cumulative cancellation. We therefore seek a complementary description that can be computed from training gradients and interpreted through their temporal geometry. The following section develops parameter displacement and consecutive-gradient cosine similarity for this purpose, connecting the latter to directional curvature and gradient noise without requiring online Hessian eigendecomposition.

III. Characterizing Constructive and Destructive Regimes

Loss spikes and oscillatory updates motivate practical diagnostics that can be computed from gradients already available during training. We develop these diagnostics by relating parameter displacement, gradient magnitudes and alignment, curvature and sampling noise. The proposed quantities characterize constructive and destructive regimes, and inform possible interventions without the additional overhead of computing Hessians.

Consider the plain SGD update in Eq. (1), with constant learning rate $\eta > 0$. As before, $\theta_t \in \mathbb{R}^p$ denotes the parameters after update t, and g_t is the mean mini-batch gradient evaluated

at θ_{t-1}. Summing the updates gives the displacement from initialization,

$$\Delta_T := \theta_T - \theta_0 = -\eta \sum_{t=1}^{T} g_t. \tag{7}$$

For a model trained from scratch, θ_0 represents the initialized model that has not yet learned from the training data. After T updates, the trained parameters θ_T encode what the model has learned within that training budget. The displacement Δ_T captures the net parameter change produced by training. Expanding the squared displacement separates gradient magnitudes from the interactions between different steps

$$\begin{aligned}\|\Delta_T\|^2 = \eta^2 \sum_{t=1}^{T} \|g_t\|^2 \\ + 2\eta^2 \sum_{1\le u<t\le T} g_u^\top g_t.\end{aligned} \tag{8}$$

The terms $\|g_t\|^2$ are nonnegative and determine the squared lengths of individual SGD steps through $\eta^2\|g_t\|^2$. In full-batch GD, a larger gradient norm indicates a steeper local loss and, therefore, a larger first-order decrease along the negative gradient. The cross-step terms $g_u^\top g_t$ describe how these steps interact – positive inner products reinforce the displacement, whereas negative inner products cancel part of it. Large gradient norms can therefore coexist with cancellation along the training trajectory, and destructive interference (i.e. persistent $g_u^\top g_t < 0$) can limit displacement growth.

To isolate directional alignment from gradient magnitudes, define the cosine similarity between any two nonzero gradients and its consecutive-step form

$$\rho_{u,t} := \frac{g_u^\top g_t}{\|g_u\|\|g_t\|}, \quad 1 \le u < t \le T. \tag{9}$$

In this setup, the alignment between consecutive gradients is $\rho_t := \rho_{t-1,t}$, $t = 2, \dots, T$. Substituting into Eq. (8) expresses the squared displacement entirely through gradient norms and directional alignment

$$\begin{aligned}\|\Delta_T\|^2 = \eta^2 \sum_{t=1}^{T} \|g_t\|^2 \\ + 2\eta^2 \sum_{1\le u<t\le T} \|g_u\|\|g_t\|\rho_{u,t}.\end{aligned} \tag{10}$$

For fixed gradient norms, positive $\rho_{u,t}$ increases the squared net displacement, negative $\rho_{u,t}$ contributes cancellation, and orthogonal gradients have no pairwise contribution. In practice, ρ_t provides an online diagnostic of the interaction between successive updates: persistent positive or negative values characterize constructive or destructive regimes.

A. Directional Curvature and Consecutive Alignment

For mini-batch SGD, we explicitly incorporate stochastic gradient noise in Eq. (4), $g_t = \nabla\mathcal{L}(\theta_{t-1}) + \xi_t$. Suppose the full objective $\mathcal{L}$ is twice continuously differentiable along the preceding step, and write $H_{t-1} := \nabla^2\mathcal{L}(\theta_{t-2})$. Expanding the full gradient around θ_{t-2} and substituting the preceding SGD update yields

$$g_t = (I - \eta H_{t-1})g_{t-1} + (\xi_t - \xi_{t-1}) + r_t, \tag{11}$$

where r_t is the Taylor remainder. Neglecting r_t recovers the first-order approximation. Taking the inner product with g_{t-1} then gives

$$\begin{aligned}\|g_{t-1}\|\|g_t\|\rho_t \approx \quad & \underbrace{\|g_{t-1}\|^2}_{\text{Gradient magnitude}} \\ & \underbrace{- \eta g_{t-1}^\top H_{t-1} g_{t-1}}_{\text{Curvature contribution}} \\ & \underbrace{+ g_{t-1}^\top(\xi_t - \xi_{t-1})}_{\text{Noise contribution}}.\end{aligned} \tag{12}$$

Positive curvature reduces the component of the next gradient along the preceding direction. Its strength relative to the preceding squared norm is the directional curvature

$$\kappa_t := \frac{g_{t-1}^\top H_{t-1} g_{t-1}}{\|g_{t-1}\|^2}, \tag{13}$$

Thus, κ_t measures how strongly a step changes the gradient along the direction actually followed, whereas ρ_t measures the resulting alignment. Sampling modifies this response through the normalized noise interaction

$$\varepsilon_t := \frac{g_{t-1}^\top(\xi_t - \xi_{t-1})}{\|g_{t-1}\|^2}. \tag{14}$$

Dividing Eq. (12) by the two gradient norms gives

$$\rho_t \approx \frac{\|g_{t-1}\|}{\|g_t\|}(1 - \eta\kappa_t + \varepsilon_t). \tag{15}$$

In full-batch GD, $\varepsilon_t = 0$. Within this first-order approximation, $\eta\kappa_t < 1$ retains a positive projection, while $\eta\kappa_t > 1$ reverses it: the next update partially opposes the preceding step. At $\eta\kappa_t = 1$, the new gradient has lost its component along the old direction. It is orthogonal if nonzero; if it vanishes, cosine is undefined. The norm ratio determines how this projected change translates into the magnitude of ρ_t.

The threshold at one also has an optimization interpretation. Along the full-gradient direction, the local quadratic loss model is $\mathcal{L}(\theta_{t-2} - \alpha g_{t-1}) \approx \mathcal{L}(\theta_{t-2}) - \alpha\|g_{t-1}\|^2 + \alpha^2\kappa_t\|g_{t-1}\|^2/2$. For $\kappa_t > 0$, its directional minimum is at $\alpha = 1/\kappa_t$. Negative alignment therefore indicates that the step has passed this directional minimum under the approximation. Other gradient components may remain: this threshold identifies directional overshoot, rather than a stationary point or the loss-increase boundary derived below.

This interpretation leads to a measurable diagnostic. Rearranging Eq. (15) gives

$$\eta\widehat{\kappa}_t := 1 - \rho_t\frac{\|g_t\|}{\|g_{t-1}\|} \approx \eta\kappa_t - \varepsilon_t. \tag{16}$$

The left-hand side uses only consecutive gradients: it measures their finite-step projected change without a Hessian evaluation. In full GD it estimates directional curvature when the Taylor remainder is small; in SGD it also contains sampling noise. It characterizes the realized step, whereas Eq. (3) concerns perturbations along the most curved local direction.

B. Noise Contribution under Mini-Batch Sampling

Unbiased sampling does not make the noise interaction vanish. Write $g_{t-1} = \nabla\mathcal{L}(\theta_{t-2}) + \xi_{t-1}$. Fresh, conditionally unbiased batches give $\mathbb{E}[g_{t-1}^\top \xi_t] = 0$ and $\mathbb{E}[\nabla\mathcal{L}(\theta_{t-2})^\top \xi_{t-1}] = 0$. With finite second moments, the preceding gradient contributes its own noise energy:

$$\mathbb{E}[g_{t-1}^\top(\xi_t - \xi_{t-1})] = -\mathbb{E}[\|\xi_{t-1}\|^2] \le 0. \tag{17}$$

Thus, even unbiased sampling noise contributes a systematic nonpositive term to the expected unnormalized interaction between consecutive gradients.

This contribution is directly related to mini-batch size. Consider a mini-batch of b examples sampled uniformly without replacement from $N > 1$ training examples. Using the per-sample gradient covariance $\Sigma(\theta)$ defined in Eq. (5), the sampling variance satisfies

$$\mathbb{E}[\|\xi_{t-1}\|^2] = \frac{N-b}{b(N-1)}\mathbb{E}[\operatorname{tr}\Sigma(\theta_{t-2})]. \tag{18}$$

At fixed parameters, smaller batches produce larger sampling variance and, consequently, a larger negative expected noise contribution. For full-batch gradients ($b = N$), this contribution vanishes.

Noise can weaken directional coherence by enlarging gradient norms relative to their shared signal. For independent estimates at two fixed states, let μ, ν be the nonzero full gradients and v_μ, v_ν the expected squared noise norms. When sampled norms concentrate,

$$\mathbb{E}[\cos(\mu+\xi, \nu+\zeta)] \approx \frac{\cos(\mu,\nu)}{\sqrt{(1+v_\mu/\|\mu\|^2)(1+v_\nu/\|\nu\|^2)}}.$$

This attenuation moves either sign toward zero. At the same fixed state, the common full-gradient direction gives the special case

$$\mathbb{E}[\rho_t] \approx \frac{\|\nabla\mathcal{L}(\theta)\|^2}{\|\nabla\mathcal{L}(\theta)\|^2 + \frac{N-b}{b(N-1)}\operatorname{tr}\Sigma(\theta)}. \tag{19}$$

Here the expected cosine is nonnegative; the approximation quantifies the dilution of a common direction by sampling. During SGD, a sampled update also changes the next state, coupling noise and curvature. The fixed-state attenuation argument therefore isolates a mechanism, while the batch-size intervention in Section IV-D tests its expression along evolving trajectories.

C. Normalized Displacement and Cumulative Interference

While the consecutive cosine similarity ρ_t measures local lag-1 directional alignment, it does not capture interactions between more distant updates (lag$\ge$ 2). To measure their cumulative effect, we define the normalized displacement

$$R_T := \frac{\|\Delta_T\|}{\eta\sum_{t=1}^T \|g_t\|} = \frac{\|\sum_{t=1}^T g_t\|}{\sum_{t=1}^T \|g_t\|}, \qquad 0 \le R_T \le 1, \tag{20}$$

For constant-step plain SGD, the denominator represents the total distance traveled in parameter space, while the numerator measures the net displacement from initialization. Thus, R_T quantifies the fraction of accumulated step length retained as net displacement.

Using Eq. (10), this ratio satisfies

$$R_T^2 = \frac{\sum_{t=1}^T \|g_t\|^2 + 2\sum_{1\le u<t\le T}\|g_u\|\|g_t\|\rho_{u,t}}{\left(\sum_{t=1}^T \|g_t\|\right)^2}. \tag{21}$$

Unlike ρ_t, which considers only adjacent gradients, R_T incorporates interactions across all temporal lags. When R_T is near one, almost all traveled distance survives as net displacement; when it is near zero, little does. Repeated back-and-forth steps and weakly coherent sampled directions both suggest low retention. For example, equal-length orthogonal updates give $R_T = 1/\sqrt{T}$, whereas equal opposing pairs cancel exactly. We therefore expect small R_T when such behavior persists over a substantial part of an SGD path. Its value is relative to traveled distance: absolute displacement can still grow, and the sign of the complete interaction in Eq. (21) requires the full decomposition.

D. Directional Overshoot and Loss Increase

Directional overshoot need not undo the loss decrease already made along a step. Define its full-objective change as $\delta\mathcal{L}_{t-1} := \mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2})$. The second-order approximation derived in Appendix A separates three contributions:

$$\begin{aligned}\delta\mathcal{L}_{t-1} \approx\ & \underbrace{-\frac{\eta}{2}\|g_{t-1}\|^2}_{\text{Gradient magnitude}} \\ & \underbrace{-\frac{\eta}{2}\rho_t\|g_{t-1}\|\|g_t\|}_{\text{Directional interference}} \\ & + \underbrace{\frac{\eta}{2}g_{t-1}^\top(\xi_{t-1}+\xi_t)}_{\text{Noise interaction}}.\end{aligned} \tag{22}$$

The magnitude term favors descent. The interference term reinforces it for positive alignment and opposes it for negative alignment: destructive interference contributes positively to the predicted loss change. With negligible noise and nonzero gradients, the approximation predicts a loss increase precisely when

$$\rho_t < -\frac{\|g_{t-1}\|}{\|g_t\|}. \tag{23}$$

This requires both negative alignment and gradient-norm growth. Neither alone suffices: the opposing component of the new gradient must exceed the preceding gradient magnitude. Large norms can therefore accompany descent, just as negative cosine can accompany a smaller loss decrease.

Substituting Eq. (15) with zero noise gives

$$\delta\mathcal{L}_{t-1} \approx \frac{\eta}{2}\|g_{t-1}\|^2(\eta\kappa_t - 2). \tag{24}$$

The two thresholds distinguish overshooting the directional minimum ($\eta\kappa_t > 1$) from overshooting far enough to raise the loss ($\eta\kappa_t > 2$). Between them, the gradients oppose each other while the step still improves the objective. These are local quadratic predictions; finite-step nonlinear effects and the explicit SGD noise term can change the observed sign. A transient increase becomes a loss peak only if subsequent updates recover.

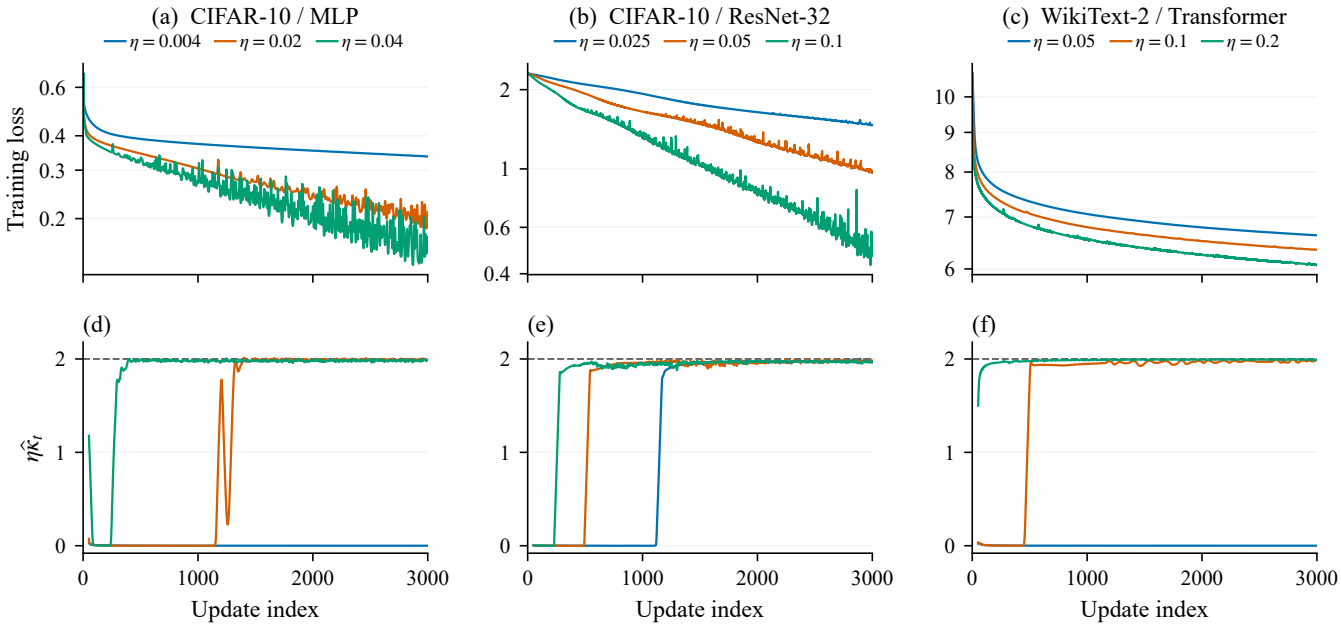


Fig. 1: Training loss and projected gradient change under full-batch GD. Columns identify the dataset/model; colors identify learning rates within each column. Top: unsmoothed full-objective loss on logarithmic axes. Bottom: complete 50-pair trailing means of the pointwise $\eta\widehat{\kappa}_t$, with the local quadratic loss-change boundary at two. Loss at index t is evaluated at θ_t; gradient diagnostics use g_t at θ_{t-1}. Each model uses one shared initialization. Smoothing summarizes the regime and does not identify its exact onset.

IV. Empirical Characterization of Gradient Interference

We use full-batch GD to characterize directional overshoot, its effect on cumulative motion, and its coexistence with loss reduction. We then vary batch size to examine how sampling changes these signatures.

The comparison covers a tanh MLP on 5,000 CIFAR-10 images, ResNet-32 on all 50,000 CIFAR-10 training images, and a word-level Transformer on WikiText-2. Each minimizes its complete fixed training objective for 3,000 updates at three constant learning rates (Table I). Within each model, rates share one frozen initialization and data preparation; they were selected by earlier training calibrations to expose contrasting dynamics, without test-loss selection. Plain GD uses no momentum, weight decay, clipping or schedule. These comparisons test whether the same geometric signatures recur across the three objectives; they do not isolate architecture effects. Appendix C specifies objectives, architectures, precision and measurement details.

A. A Gradient-Based View of Oscillatory Descent

Figure 1 pairs loss with the projected gradient change $\eta\widehat{\kappa}_t$. The higher-rate trajectories develop oscillatory loss while continuing to improve over longer timescales; their late projected change approaches two. Equation (24) explains why this value is informative: in the local quadratic model, it separates a loss-decreasing step from one that reverses the preceding progress. The observed pattern gives a gradient-based view of the oscillatory descent associated with the Edge of Stability [13].

The diagnostic requires consecutive gradients rather than Hessian–vector products or an eigensolver. It estimates curvature along the realized step when the local approximation applies; it does not estimate the largest Hessian eigenvalue that defines spectral EoS. This distinction permits a practical interpretation without treating the two quantities as interchangeable. The same local model explains the regime but does not reliably predict every nonlinear loss fluctuation: the stepwise approximation check and spectral coverage are retained in Appendix C.

B. Reversal, Step Magnitude and Displacement Retention

Figure 2 shows how the oscillatory regime changes parameter motion. Persistent negative cosine means that successive updates repeatedly oppose the preceding direction, consistent with overshooting its local directional minimum. The lowest MLP and Transformer rates retain positive alignment, whereas the other seven conditions approach nearly opposing gradients. In the MLP and ResNet, the transition also accompanies growing gradient norms: the steps become larger even as their directions increasingly cancel. These fluctuations arise without mini-batch sampling.

The decreasing R_t shows that larger steps need not translate into proportionally larger net motion. Repeated reversals consume path length, so the retained fraction can become small even while parameters continue to move away from initialization. This is a geometric inefficiency, not evidence of slower optimization. Table I makes the distinction concrete: within every model, the highest tested rate has the lowest

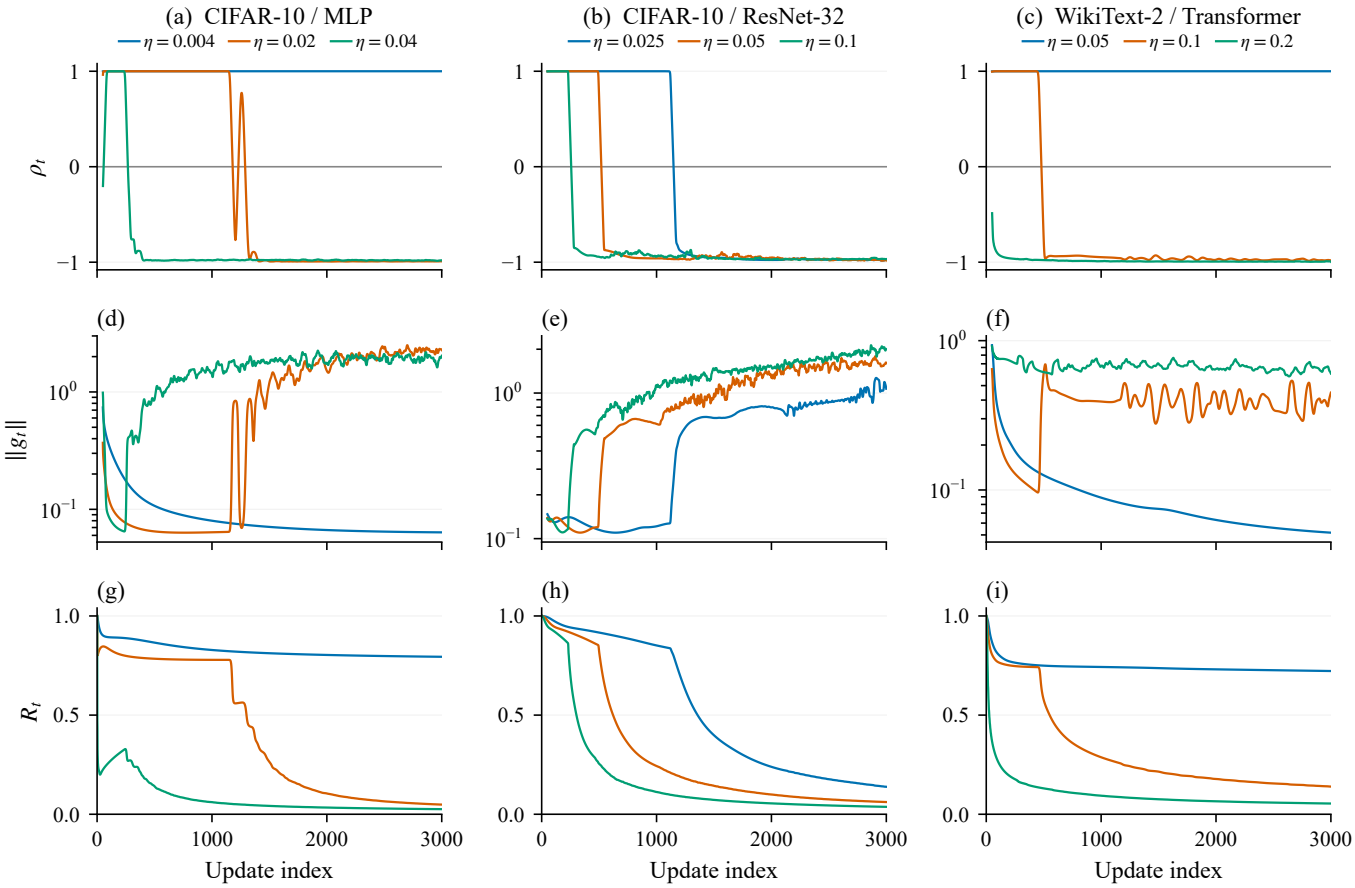


Fig. 2: Directional coherence and cumulative motion for the conditions in Fig. 1. Rows show consecutive cosine, gradient norm (logarithmic scale), and normalized displacement. Cosines and norms use complete 50-observation trailing means; R_t is the unsmoothed ratio over the complete prefix through update t. Colors and column ordering match Fig. 1. Raw cosine traces appear in Appendix C.

retention, the largest absolute displacement, and the lowest final training loss. A rate can therefore be large relative to local directional minima yet productive over the full training horizon.

Adjacent cancellation also does not determine the sign of the complete cross-step interaction. That total is positive at the endpoint in all nine trajectories, including the seven with negative accumulated lag-one interaction (Table IV). Longer-range reinforcement can coexist with repeated local reversal. Together, ρ_t, gradient norms and R_t distinguish local overshoot, step size and retained motion; loss establishes whether this motion improves the objective.

C. Oscillation Beyond Adjacent Updates

To examine the temporal extent of reversal, we fix one endpoint gradient and compare it with preceding gradients at every lag $k = 1, \ldots, 256$. Figure 3 uses the largest tested rate for each model and the common endpoint after 2,256 updates. Near-zero similarity at some lags does not imply that directional memory has disappeared: strongly opposing and reinforcing pairs can reappear at longer separations. The varying amplitude and sign pattern show that oscillation extends beyond a single adjacent pair, while its persistence depends on the model and part of the trajectory. This temporal structure explains why a lag-one statistic cannot summarize the complete interference term.

TABLE I: Endpoints at $T = 3000$ updates. The late means $\overline{\rho}_{\text{late}}$ and $\overline{\eta\widehat{\kappa}}_{\text{late}}$ average gradient indices $t = 2001, \ldots, 3000$. Displacement is $\|\Delta_T\| = \eta\|\sum_{t=1}^{T} g_t\|$, subject to parameter-update roundoff. Each model shares one initialization across rates; comparisons are within models. *Positive value below 0.00005, rounded to zero.

Dataset	Model	η	$\mathcal{L}(\theta_T)$	$\overline{\rho}_{\text{late}}$	R_T	$\overline{\eta\widehat{\kappa}}_{\text{late}}$	$\|\Delta_T\|$
CIFAR-10	MLP	0.004	0.3365	1.0000	0.7941	0.0000*	0.9511
		0.02	0.2114	-0.9913	0.0488	1.9924	3.1391
		0.04	0.1539	-0.9799	0.0257	1.9820	4.7628
	ResNet-32	0.025	1.4665	-0.9723	0.1385	1.9740	5.7045
		0.05	0.9672	-0.9708	0.0616	1.9746	9.0183
		0.1	0.4898	-0.9684	0.0374	1.9707	13.9329
WikiText-2	Transformer	0.05	6.6301	1.0000	0.7223	0.0002	11.3183
		0.1	6.3519	-0.9749	0.1395	1.9754	15.6026
		0.2	6.0697	-0.9937	0.0541	1.9937	21.9693

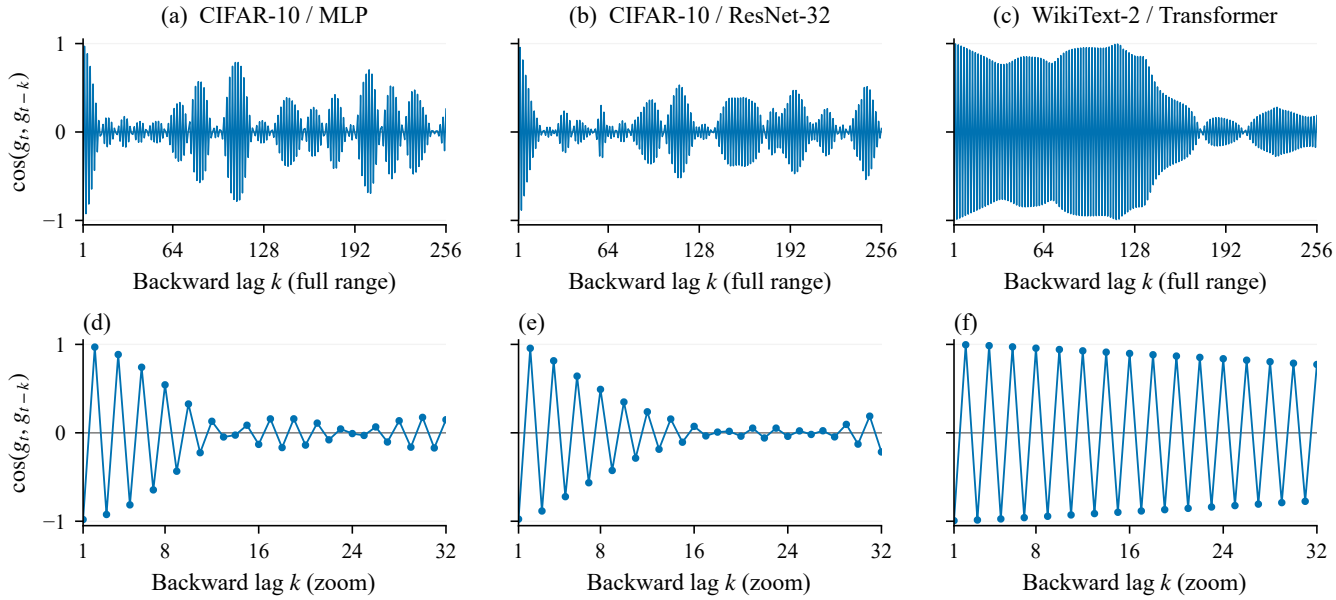


Fig. 3: Backward gradient alignment at the common endpoint state θ_{2256}, corresponding to $t = 2257$ under the manuscript's preupdate gradient indexing. Each point is one exact cosine $\cos(g_t, g_{t-k})$; the endpoint is fixed across all 256 lags. The top row shows lags 1–256; the bottom row magnifies the same values at lags 1–32 to resolve the early sign alternation. Rates are 0.04 (MLP), 0.1 (ResNet-32) and 0.2 (Transformer). These are separately recomputed 256-update GD continuations from retained state 2000, with one initialization and no smoothing or endpoint averaging. Appendix C records the additional endpoints and numerical replay scope.

D. Mini-Batch Noise and Directional Coherence

We vary batch size at initialization and after 2,000 GD updates in the same three model–dataset cases, holding $\eta = 0.02$, 0.1 and 0.2 fixed for the MLP, ResNet-32 and Transformer, respectively. Each starting state receives 200-update continuations, with three initializations, eight sampling repetitions per stochastic condition and one full-GD control per initialization. Figure 4 compares mean sampled consecutive-gradient cosine over each continuation. Equal update budgets imply unequal data exposure; sampling units, objective coverage and numerical checks are in Appendix C.

The smallest batches weaken trained-state reversal in all three cases. Initial-state responses are less uniform: full GD has positive mean cosine in the MLP and ResNet and strongly negative mean cosine in the Transformer, where reversal develops within the first 200 updates. The MLP's sampled paths can also acquire negative mean alignment while its full-GD mean is positive. Sampling therefore changes the regime encountered along a path, in addition to perturbing gradients at fixed parameter states.

For the Transformer at $b = 8$, both starting states have mean cosine near -0.26, yet mean full-data loss falls from initialization and rises after GD training. Mean adjacent alignment alone therefore does not order training-objective progress.

The MLP's smallest trained-state batch likewise raises mean full-data loss, whereas full GD lowers it in both models.

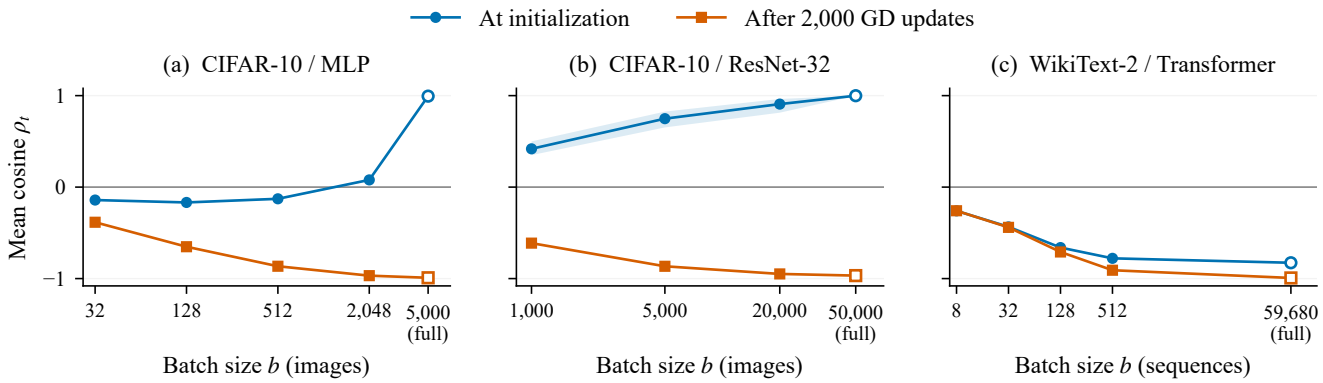


Fig. 4: Mean sampled-gradient cosine over 199 adjacent pairs during 200-update continuations in three model–dataset cases. Blue circles start at initialization; orange squares start after 2,000 GD updates. Each line averages repetitions within initialization, then weights the three initialization blocks equally; shading is their range, not a confidence interval. Hollow rightmost markers denote full GD. ResNet batches contain whole fixed 1,000-image normalization groups; Transformer batches count prepared sequences. Batch axes are logarithmic; lines guide the eye between measured sizes.

Noise covariance can affect the solutions favored by training [28], but mean cosine alone identifies neither that bias nor a generalization benefit. These comparisons describe selected objectives, rates and update budgets; they do not select an optimal batch size.

V. Directional Diagnostics under Adam

The geometric diagnostics developed for SGD can also characterize training under Adam, provided we distinguish the gradient information from the parameter motion produced by the optimizer. Raw gradients describe the instantaneous stochastic loss signal along the training trajectory; momentum averages this signal over time; and adaptive preconditioning determines how the accumulated information becomes a parameter update. We apply the same diagnostics to these successive quantities without transferring the SGD stability conditions to Adam.

A. Three Views of the Same Training Trajectory

For Adam [16], the first and second moments satisfy

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}, \end{aligned} \tag{25}$$

with zero initial moments. Let $\widehat{m}_t = m_t/(1-\beta_1^t)$ and $\widehat{v}_t = v_t/(1-\beta_2^t)$ denote their bias-corrected values. The adaptive direction and parameter update are

$$d_t = \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}, \qquad \theta_t = \theta_{t-1} - \eta_t d_t, \tag{26}$$

where squaring, square roots and division act coordinatewise. Here $\epsilon > 0$ is Adam's numerical constant, distinct from the sampling contribution ε_t in Section III. If gradients are clipped, the clipped vector replaces g_t in Eq. (25); raw-gradient diagnostics still use the unclipped gradient. AdamW additionally applies decoupled weight decay [29].

The norms, consecutive cosines and accumulation ratio from Section III can be computed for each of g_t, m_t and d_t. We identify the vector stream explicitly rather than redefine each diagnostic. Raw gradients reveal changes in the sampled loss signal; first moments show what temporal averaging retains; adaptive directions show the effect of coordinate scaling on that retained signal. All three are available within the optimizer, although measuring consecutive alignment requires retaining a previous vector, and cumulative ratios require a running vector sum. Their computation uses reductions rather than new backward passes; storage and communication costs still scale with the measured parameter scope.

An accumulation ratio computed from raw gradients or moments is not normalized parameter displacement under Adam. To recover that quantity, apply the same norm-of-sum over sum-of-norms construction to the actual parameter increments. Even for Adam without decay, these increments include η_t: varying the learning rate changes their relative weights. AdamW decay adds a separate displacement, and parameter-group learning rates or finite-precision updates can change the correspondence further. Positive scalar bias correction preserves consecutive momentum cosine, but generally changes cumulative accumulation because the scalar varies with time.

B. Raw Gradients as Observations of the Traversed Loss

Raw-gradient alignment remains informative under Adam because it records how the loss gradient changes along the optimizer's chosen path. The Taylor argument in Section III can be written using the actual preceding displacement:

$$\begin{aligned} g_t - g_{t-1} \approx H_{t-1}(\theta_{t-1} - \theta_{t-2}) \\ + \xi_t - \xi_{t-1}. \end{aligned} \tag{27}$$

This uses the same Hessian indexing and neglected remainder as Eq. (11). Under plain Adam, the displacement is $-\eta_{t-1} d_{t-1}$, rather than a scaled raw gradient. Consequently, the observed response combines local curvature, the direction selected by the optimizer and sampling variation. The SGD proxy in Eq. (16) remains computable as a normalized

gradient-change statistic, but its directional-curvature interpretation does not carry over, even if sampling noise is small.

A return to positive raw-gradient cosine after reducing the learning rate therefore describes a change in the sampled gradient response. It does not by itself show that curvature decreased: shorter steps can change that response within the same local quadratic geometry. Nor does negative raw alignment imply that consecutive Adam updates oppose each other. That question must be answered from the transformed directions or actual parameter increments.

C. Momentum Growth, Rotation and Adaptive Scaling

The first-moment recurrence separates two effects of an incoming gradient. Its component along the stored moment can reinforce or oppose the current momentum, while its transverse component changes the direction. Increasing the magnitude of an aligned input can grow the moment without appreciable rotation. Increasing a transverse input relative to the stored moment can instead lower consecutive momentum cosine while that cosine remains positive. Thus, a decline from near one already indicates greater angular change; a sign reversal is not required.

Positive alignment between consecutive raw gradients does not guarantee that the latest gradient aligns with the longer history stored in m_{t-1}. Interpreting momentum rotation therefore benefits from also comparing the incoming optimizer gradient with the previous moment. This is a distinct comparison from adjacent raw-gradient cosine. With clipping, the magnitude entering the recurrence is the clipped magnitude: growth of the raw norm beyond the threshold need not produce corresponding moment growth. The signs and norms of the recorded streams alone do not identify that propagation mechanism.

Preconditioning adds another transformation. The same first moment can produce different update directions as coordinate weights change, and a large moment need not imply a large adaptive step. Comparing momentum and adaptive-direction alignment reveals where their temporal behavior differs, but their numerical difference is not an additive measure of the preconditioner's effect. Likewise, a highly aligned adaptive sequence can still make poor objective progress. Loss measurements and controlled direction or step-size interventions are needed to connect these geometric descriptions to optimization outcomes.

D. Illustrations in Language-Model Training

We illustrate these distinctions with small Pythia-family decoder models [30], initialized from scratch and trained on MiniPile [31]. A seven-rate comparison uses one shared 14M initialization and data order (seed 1234), with linear warmup over the first 1% of training followed by a constant rate. Each run processes the same complete 1.491-billion-input-token cache in 712 updates, including the final partial batch. Full batches contain 1,024 packed sequences of length 2,048, accumulated in microbatches of four. AdamW uses $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$, global gradient clipping at norm one, and decay 0.1 excluding biases and layer-normalization parameters. Computation is FP16 with FP32 parameters and moments; diagnostic reductions are FP64. Loss is averaged over supervised next-token targets. Validation covers all 500 held-out documents and 693,329 targets, including the packed tail, with token weighting.

TABLE II: Pythia-14M: median consecutive cosine over updates 612–711 and final validation loss. Raw, moment and adaptive columns refer to g_t, m_t and d_t, respectively. One initialization; rows are grouped by execution platform. Rates are rounded for display.

Rate	Raw	Moment	Adaptive	Val. loss
DGX Spark (GB10)				
10^{-4}	-0.334	0.867	0.978	6.232
10^{-3}	-0.432	0.916	0.942	4.764
10^{-2}	-0.103	0.403	0.854	6.636
RTX 4090				
2.15×10^{-4}	-0.434	0.573	0.972	5.601
4.64×10^{-4}	-0.495	0.762	0.958	5.238
2.15×10^{-3}	-0.310	0.956	0.912	4.410
4.64×10^{-3}	-0.361	0.852	0.854	4.318

TABLE III: Checkpoint-matched schedule intervention. Median raw cosine uses affected pairs 358–712; loss is final full-validation loss. Each size uses one initialization (seed 1235).

	14M		70M	
Schedule	Raw cosine	Val. loss	Raw cosine	Val. loss
Original decay	0.194	5.182	-0.109	4.056
Held rate	-0.358	5.067	-0.311	3.985
Faster decay	0.331	5.218	-0.006	4.104

Table II shows negative late raw-gradient alignment alongside positive first-moment and adaptive alignment at every tested rate. No recorded consecutive first-moment pair is negative in these seven runs, yet its late median falls to 0.403 at rate 10^{-2}. That run clips 517 of 712 updates and has late median raw norm 5.566, while the moment norm is only 0.097. These measurements illustrate the distinction between large raw gradients and the signal accumulated after clipping and averaging; they do not establish raw-norm growth as the cause of momentum rotation. Within the hardware-matched RTX 4090 runs, rate 4.64×10^{-3} has lower momentum alignment than 2.15×10^{-3} but better validation loss. The comparison is descriptive: it uses one initialization, and pooled differences across platforms cannot be attributed solely to learning rate.

A separate intervention branches 14M and 70M AdamW checkpoints at update 356 into three schedules: continue the original cosine decay, hold its current rate, or advance its decay clock twice as fast. Within each size, weights, optimizer state and future batches are matched; all branches continue through update 712 on RTX 5090 with the same recipe and validation coverage. The held rate is 5.1791×10^{-4}; original and accelerated decay reach the 10^{-4} floor at updates 575 and 466. Table III shows that faster decay makes raw alignment less negative, while held-rate continuations attain lower validation loss. Positive smoothed alignment also occurs earlier under accelerated decay, but later reverses in the 70M model. Changing the rate also changes AdamW's decay displacement, so these interventions identify schedule effects without isolating their mediation by step length or shrinkage.

E. From Characterization to Informed Scheduling

These observations motivate using the diagnostics to formulate and evaluate learning-rate interventions. Raw-gradient reversal with coherent momentum indicates that averaging is retaining a direction despite local variation. A simultaneous rise in raw norms, frequent clipping, declining momentum alignment and worsening loss supplies a different pattern that merits a step-size comparison. A change concentrated in adaptive directions calls for examining coordinate scaling and actual step lengths. These are interpretive patterns, not validated trigger thresholds.

An informed scheduling rule should therefore be evaluated by its subsequent loss improvement at a matched budget, rather than by whether it maximizes cosine similarity or displacement retention. The next test is to branch shared checkpoints into predefined rate interventions, record recent gradient, momentum and adaptive behavior, and compare later outcomes on held-out data across initializations. Such a test must establish whether the diagnostics add information beyond the current rate, training stage, clipping frequency and recent loss. The present extension supplies the measurement and interpretation framework for that test; it does not yet establish an improved adaptive schedule.

VI. Conclusion

The displacement formulation distinguishes the magnitude of individual updates, their local directional interaction and their cumulative accumulation. For plain SGD, the exact decomposition retains all temporal lags, while consecutive cosine provides a tractable local observation. The curvature and sampling relations clarify how that observation can change without determining either cumulative displacement or loss improvement. The full-batch and mini-batch experiments illustrate these distinctions: gradient reversal can coexist with objective progress, and weaker reversal need not improve displacement retention or the attained loss.

Under Adam, the same measurements acquire different meanings at successive stages of the optimizer. Raw gradients describe the sampled loss response; first moments expose temporal averaging; adaptive directions incorporate coordinate scaling. Parameter displacement additionally depends on the applied learning rates and weight decay. The language-model comparisons show why positive momentum or adaptive alignment cannot serve as a universal performance criterion, and why restoring positive raw alignment through decay need not improve the final validation loss.

The practical contribution is a framework for interpreting these measurements together and using them to formulate controlled interventions. Developing a scheduler requires a further step: demonstrating that the diagnostics improve rate decisions beyond recent loss and known training settings, at matched budgets and across independent runs. That evaluation remains future work.

Appendix A
Derivation of the Alignment-Based Loss Change

Consider the SGD update $\theta_{t-1} = \theta_{t-2} - \eta g_{t-1}$. The change in the full objective along this step can be expressed exactly as

$$\begin{aligned}&\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \\ &\quad = -\eta \int_0^1 g_{t-1}^\top \nabla \mathcal{L}(\theta_{t-2} - s\eta g_{t-1})\, ds.\end{aligned} \tag{28}$$

Approximating the integral by the average of the full gradients at the two endpoints gives

$$\begin{aligned}&\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \\ &\quad \approx -\frac{\eta}{2} g_{t-1}^\top \left(\nabla \mathcal{L}(\theta_{t-2}) + \nabla \mathcal{L}(\theta_{t-1})\right).\end{aligned} \tag{29}$$

This approximation is exact for quadratic objectives. For nonlinear objectives, the approximation error is of third order in the step length when the Hessian is locally Lipschitz continuous.

Using the stochastic gradient decomposition $g_t = \nabla \mathcal{L}(\theta_{t-1}) + \xi_t$ at both endpoints, we obtain

$$\begin{aligned}&\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \\ &\quad \approx -\frac{\eta}{2} g_{t-1}^\top (g_{t-1} + g_t - \xi_{t-1} - \xi_t) \\ &\quad = -\frac{\eta}{2} \left(\|g_{t-1}\|^2 + g_{t-1}^\top g_t\right) \\ &\qquad + \frac{\eta}{2} g_{t-1}^\top (\xi_{t-1} + \xi_t).\end{aligned} \tag{30}$$

Substituting $g_{t-1}^\top g_t = \|g_{t-1}\| \|g_t\| \rho_t$ and factoring the preceding squared gradient norm yields

$$\begin{aligned}&\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \\ &\quad \approx -\frac{\eta}{2} \|g_{t-1}\|^2 \left(1 + \rho_t \frac{\|g_t\|}{\|g_{t-1}\|}\right) \\ &\qquad + \frac{\eta}{2} g_{t-1}^\top (\xi_{t-1} + \xi_t),\end{aligned} \tag{31}$$

which recovers Eq. (22).

To connect this expression with directional curvature, recall from Eq. (15) that

$$\rho_t \frac{\|g_t\|}{\|g_{t-1}\|} \approx 1 - \eta \kappa_t + \varepsilon_t. \tag{32}$$

Substituting this relation and using the definition of ε_t in Eq. (14) gives

$$\begin{aligned}&\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \\ &\quad \approx -\eta \|g_{t-1}\|^2 \left(1 - \frac{\eta \kappa_t}{2}\right) + \eta \xi_{t-1}^\top g_{t-1}.\end{aligned} \tag{33}$$

Thus, the curvature-based and alignment-based expressions describe the same local loss change to second order. Positive directional curvature counteracts the first-order loss decrease, while its effect is already incorporated into the weighted alignment between consecutive gradients. The noise contributions account for the difference between the observed mini-batch gradients and the full gradients.

Appendix B
Derivation of the Local Loss Changes

We use the same indexing as Section III: g_{t-1} is evaluated at θ_{t-2}, g_t at θ_{t-1}, and $H_{t-1} = \nabla^2 \mathcal{L}(\theta_{t-2})$. The loss is assumed twice continuously differentiable along the displacements considered below. The relations retain second-order terms and omit the Taylor remainders; they are exact when the objective is quadratic.

A. One Update and Directional Curvature

For $\theta_{t-1} - \theta_{t-2} = -\eta g_{t-1}$, Taylor expansion gives

$$\begin{aligned}&\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \\ &\quad\approx -\eta \nabla \mathcal{L}(\theta_{t-2})^\top g_{t-1} + \frac{\eta^2}{2} g_{t-1}^\top H_{t-1} g_{t-1}.\end{aligned} \tag{34}$$

Substituting $\nabla \mathcal{L}(\theta_{t-2}) = g_{t-1} - \xi_{t-1}$ and $g_{t-1}^\top H_{t-1} g_{t-1} = \kappa_t \|g_{t-1}\|^2$ yields Eq. (22) for nonzero g_{t-1}.

The positive term $\eta \xi_{t-1}^\top g_{t-1}$ in that expression cannot be interpreted separately as the net effect of noise. Conditional on the parameters before the batch is drawn, unbiased sampling gives

$$\begin{aligned}\mathbb{E}[\|g_{t-1}\|^2 \mid \theta_{t-2}] &= \|\nabla \mathcal{L}(\theta_{t-2})\|^2 \\ &\quad + \mathbb{E}[\|\xi_{t-1}\|^2 \mid \theta_{t-2}], \\ \mathbb{E}[\xi_{t-1}^\top g_{t-1} \mid \theta_{t-2}] &= \mathbb{E}[\|\xi_{t-1}\|^2 \mid \theta_{t-2}].\end{aligned} \tag{35}$$

Thus, the contributions from noise variance cancel between the first-order terms when taking their conditional expectation. With finite moments for the retained terms, the expected loss change in the quadratic approximation is

$$\begin{aligned}&\mathbb{E}[\mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2}) \mid \theta_{t-2}] \\ &\quad\approx -\eta \|\nabla \mathcal{L}(\theta_{t-2})\|^2 \\ &\qquad + \frac{\eta^2}{2} \mathbb{E}[g_{t-1}^\top H_{t-1} g_{t-1} \mid \theta_{t-2}].\end{aligned} \tag{36}$$

Sampling still contributes to the curvature-weighted second-order term. This conditional calculation does not remove that contribution or the Taylor remainder from a nonlinear trajectory.

B. Two Updates and Consecutive Alignment

The displacement over two SGD updates is exactly

$$\theta_t - \theta_{t-2} = -\eta(g_{t-1} + g_t).$$

Expanding around the starting parameters θ_{t-2} gives

$$\begin{aligned}&\mathcal{L}(\theta_t) - \mathcal{L}(\theta_{t-2}) \\ &\quad\approx -\eta \nabla \mathcal{L}(\theta_{t-2})^\top (g_{t-1} + g_t) \\ &\qquad + \frac{\eta^2}{2} (g_{t-1} + g_t)^\top H_{t-1} (g_{t-1} + g_t).\end{aligned} \tag{37}$$

The first-order projection can be written as

$$\begin{aligned}&\nabla \mathcal{L}(\theta_{t-2})^\top (g_{t-1} + g_t) \\ &\quad= \|g_{t-1}\|^2 + g_{t-1}^\top g_t - \xi_{t-1}^\top (g_{t-1} + g_t).\end{aligned} \tag{38}$$

Finally, $g_{t-1}^\top g_t = \|g_{t-1}\| \|g_t\| \rho_t$ yields a form of Eq. (37) using consecutive-gradient cosine. The same starting Hessian appears throughout this expansion. For a nonlinear objective, its remainder is governed by the combined displacement; a small cosine alone does not control that approximation error.

APPENDIX C
MEASUREMENT DETAILS AND ADDITIONAL FULL-BATCH DIAGNOSTICS

A. Objectives and Measurement

The tanh MLP has two 200-unit hidden layers and minimizes half-squared error summed over ten output coordinates and averaged over the first 5,000 training images. ResNet-32 minimizes mean cross-entropy on all 50,000 training images. The word-level Transformer has two layers, two attention heads, and embedding/feed-forward dimensions of 200; its objective is token-weighted negative log-likelihood. Learning rates are constant and share frozen seed-0 initial tensors within each case. There is no momentum, decay, clipping, augmentation, dropout or schedule.

The MLP uses coordinate-wise standardization and ResNet uses channel-wise standardization, both based on the complete CIFAR-10 training split. ResNet's objective averages the same 50 fixed groups of 1,000 images, with batch normalization performed separately within each group. Accumulating all group gradients gives the gradient of this fixed objective; physical batching introduces no random sampling.

For the Transformer, the corpus is arranged into 20 fixed streams and causal chunks of at most 35 tokens. Eight stream-tail tokens and the 20 initial context tokens have no training target; short chunks retain their valid targets, and padding is excluded from the loss. The resulting 59,680 sequences contain 2,088,600 training targets. The 33,278-word vocabulary follows the first-occurrence dictionary over training, validation and test text in the PyTorch word-language-model example, revision `13acec6d7c78` [32]. The same pinned example supplies the Transformer implementation. These comparisons concern training dynamics rather than held-out generalization.

Physical accumulation uses 1,000-image chunks for the MLP, the fixed 1,000-image groups for ResNet, and 64-sequence chunks for the Transformer. The complete objective is accumulated before each parameter update. Training parameters and gradients are FP32, TF32 is disabled, and diagnostic reductions are FP64. The nine conditions are paired learning-rate comparisons within three model cases, not nine independent initializations or a controlled comparison of architectures. The gradient diagnostics require vector reductions and retained gradient information rather than online eigendecomposition; their runtime and memory overhead have not been isolated by a controlled benchmark.

The first adjacent pair is omitted without a denominator offset. The complete 50-observation trailing means aid visualization and do not define regime-onset times. Endpoint losses and R_{3000} are unsmoothed.

B. Unsmoothed Alignment and Temporal Accounting

Figure 5 supplies the unsmoothed consecutive cosines underlying Fig. 2(a)–(c). Complete trailing means suppress short transients and shift the displayed transition relative to the first negative pair. In particular, the MLP at $\eta = 0.04$ reverses near initialization and then returns to positive alignment before its later negative phase. These raw traces distinguish such

a transient from a persistent pattern without introducing a stability threshold or discarding early observations.

For the complete prefix of T gradients, define the traveled distance and the temporal contributions as

$$\begin{aligned} P_T &:= \eta \sum_{t=1}^{T} \|g_t\|, \\ I_T^{(1)} &:= 2 \sum_{t=2}^{T} g_{t-1}^\top g_t, \\ I_T^{(>1)} &:= 2 \sum_{\substack{1 \le u < t \le T \\ t-u > 1}} g_u^\top g_t, \\ I_T &:= I_T^{(1)} + I_T^{(>1)} = \Big\| \sum_{t=1}^{T} g_t \Big\|^2 - \sum_{t=1}^{T} \|g_t\|^2. \end{aligned} \tag{39}$$

Table IV reports these quantities at $T = 3000$. The interaction columns use raw gradients and include the factor of two; they are not multiplied by η^2. The total is positive in all nine conditions, whereas the adjacent contribution is negative in seven. This endpoint result does not assert positivity at every prefix or for every individual pair. Comparisons of magnitudes are within each model case and its fixed parameterization.

C. Retrospective Loss Approximation

For each gradient pair indexed by $t = 2, \ldots, 3000$, let $\delta\mathcal{L}_{t-1} := \mathcal{L}(\theta_{t-1}) - \mathcal{L}(\theta_{t-2})$. We compare this observed change with the gradient-based approximation

$$\widehat{\delta\mathcal{L}}_{t-1} := -\frac{\eta}{2} \|g_{t-1}\|^2 \left(2 - \eta \widehat{\kappa}_t\right). \tag{40}$$

The normalized absolute discrepancy is

$$e_t := \frac{|\delta\mathcal{L}_{t-1} - \widehat{\delta\mathcal{L}}_{t-1}|}{\eta \|g_{t-1}\|^2}. \tag{41}$$

The denominator is the magnitude of the first-order full-GD descent term. It is nonzero for every measured pair. Table V reports the median over all 2,999 pairs and the count of opposite signs. The final update is excluded because its subsequent gradient was not recorded. All inputs are unsmoothed; the table uses no seed aggregation.

For the threshold sensitivity columns, define

$$h_t(\tau) := \tau \max\{|\mathcal{L}(\theta_{t-2})|, |\mathcal{L}(\theta_{t-1})|\}. \tag{42}$$

A transition is eligible only when both $|\delta\mathcal{L}_{t-1}| > h_t(\tau)$ and $|\widehat{\delta\mathcal{L}}_{t-1}| > h_t(\tau)$. Each cell prints opposite-sign transitions divided by its eligible count; the median discrepancy remains computed over all pairs. These thresholds are descriptive sensitivity choices, not calibrated FP32 error bounds. Serial transitions are not independent initialization replicates, and the comparison is retrospective because it uses the subsequent gradient. The discrepancy does not isolate finite-step, nonsmooth or numerical effects. For the highest-rate Transformer, the unthresholded signs disagree in 44.2% of transitions; excluding changes below the 10^{-5} relative threshold leaves 43.7% disagreement. Thus a projected change near two is informative about the oscillatory regime but does not by itself validate a pointwise nonlinear loss prediction.

D. Backward-Lag Measurements

The backward profiles use the largest GD rate in each model, with exact full-objective gradients from separately recomputed 256-update continuations. Retained states 500, 2000 and 3000 supply common starting parameters; their endpoints are states 756, 2256 and 3256 (gradient indices 757, 2257 and 3257). Figure 3 uses the middle endpoint in every model, a common trained-state comparison rather than an endpoint chosen for maximum oscillation. Each curve holds that endpoint fixed, includes every lag through 256, and uses no temporal averaging or compression. FP64 products of FP32 vectors were checked independently on CPU and GPU; selected retained vectors supply additional local checks. Later floating-point trajectories can depart from earlier replays despite identical starting tensors, so these profiles are separate continuations, not reconstructed pairs from Figs. 1 and 2. At all nine measured endpoints, both positive and negative similarities occur within lags 193–256; their amplitudes vary substantially across endpoints.

E. Mini-Batch Comparison

The mini-batch comparison uses three initializations for each objective. The MLP reuses the completed 5,000-image experiment at $\eta = 0.02$, with batches $32, 128, 512, 2048, 5000$. Fresh 2,000-update GD references at $\eta = 0.1$ and 0.2 supply ResNet and Transformer starting states. ResNet samples $1, 5, 20$ or all 50 fixed 1,000-image normalization groups, preserving each image's normalization context; its figure axis reports image counts. Transformer batches contain $8, 32, 128, 512$ or all $59{,}680$ prepared sequences, with $M = 2{,}088{,}600$ supervised tokens in the complete training objective. If ℓ_i is sequence i's summed token loss, the sampled estimator is $\frac{N}{bM} \sum_{i \in B} \ell_i$, with $N = 59{,}680$; a random sampled-token denominator would change this estimator. Every stochastic batch is a uniform subset without replacement, independently redrawn at each update. Batch sizes share prefixes of a common permutation within a repetition; permutations are independent across updates and repetitions. Parameters and gradients use FP32, reductions use FP64, and TF32 is disabled. Full-loss evaluations cover every training image or supervised token. Cosines average all 199 defined consecutive pairs; retention uses all 200 applied gradients. Eight repetitions are averaged within initialization before the three initializations receive equal weight. The plotted range represents variation among initialization-block means, not sampling confidence. Cases differ in objective, architecture, data size and selected learning rate; their contrasts do not isolate any one factor.

Cloud runs use PyTorch 2.8/CUDA 12.8. The Transformer uses RTX5090 and 512-sequence physical accumulation, without changing effective batches. ResNet uses L40S/RTX4090; seed 2's smallest two batches use GB10/PyTorch 2.11/CUDA 13 from hash-identical cloud checkpoints. Complete-gradient and short replay checks qualify this runtime; a 200-update initial full-GD bridge has endpoint parameter relative error 2.36×10^{-7} versus the cloud. This does not establish long-trajectory replay. Batch and runtime effects are confounded in

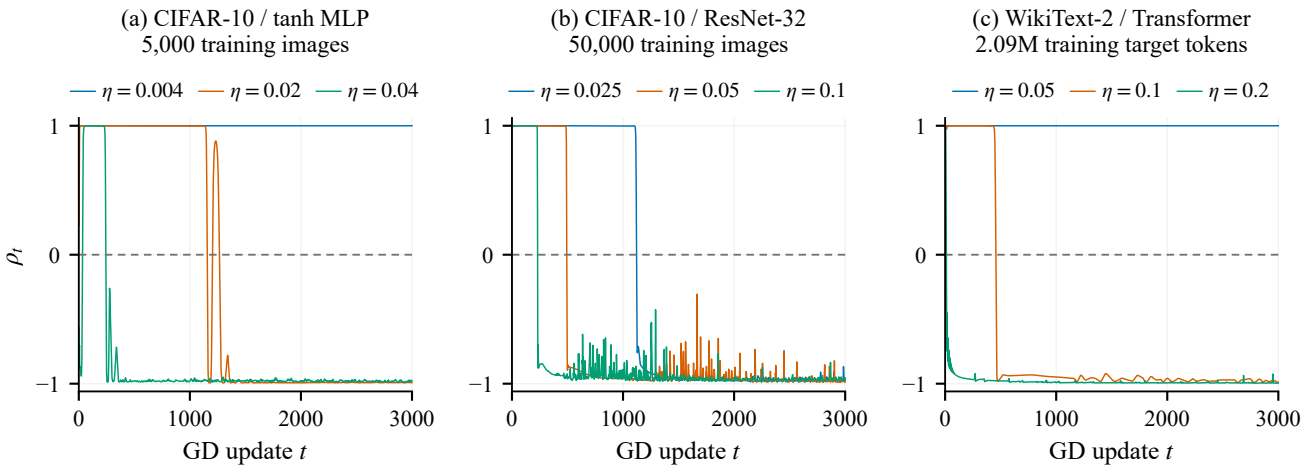


Fig. 5: Unsmoothed consecutive-gradient cosine for the nine full-GD conditions in Section IV, with 2,999 defined pairs per condition. Columns show (a) MLP, (b) ResNet-32 and (c) Transformer. The dashed horizontal reference is zero. Blue, orange and green identify increasing learning rates within each case, as given in the legends. Every curve uses seed 0 and covers the same 3,000-update horizon; the first undefined pair is omitted and there is no temporal or seed averaging. Coincident positive curves can overlap exactly.

this block; shading includes initialization and runtime variation. Initial full controls for ResNet seed 2 and Transformer seed 0 reuse the exact first 200 reference updates, with separate endpoint losses; other controls execute independently. Audits of actual increments and endpoint displacement quantify FP32 update roundoff.

F. Spectral Context

We measure the largest algebraic Hessian eigenvalue at states $0, 100, \ldots, 3000$ using matrix-free Hessian–vector products and a checked relative eigenpair residual of at most 0.005. The Hessian objective covers all 5,000 MLP training examples, but only the first five fixed ResNet groups and the first 2,500 Transformer sequences. Subset measurements follow the computational motivation of [13]; they are not full-objective sharpness for the larger models. Reused MLP trajectories requested two largest eigenvalues and the other runs requested one; we compare only the largest value. Table VI retains all nine final measurements.

Maximum pointwise eigenvalue concerns the most curved local direction, whereas $\widehat{\kappa}_t$ concerns realized gradient change across a step. Objective coverage and a finite-step Taylor remainder also affect their comparison. Final spectral values and the late means in Table I use different temporal reductions. Their agreement or disagreement therefore does not isolate estimator accuracy or the cause of a discrepancy. A state-matched local comparison would pair the Hessian at state s with the expansion for gradient index $t = s + 2$; the spectrum at state 3000 has no matching recorded gradient pair.

TABLE IV: Complete-prefix temporal accounting at $T = 3000$, with the quantities defined in Eq. (39). P_{3000} is path length; the last three columns are raw-gradient interactions. All nine conditions use one shared initialization per case. Values are rounded only for display, so displayed components may not sum exactly to the displayed total.

Model	η	P_{3000}	$I^{(1)}_{3000}$	$I^{(>1)}_{3000}$	I_{3000}
MLP	0.004	1.1976	1.270×10^{2}	5.634×10^{4}	5.647×10^{4}
MLP	0.02	64.3608	-1.217×10^{4}	3.063×10^{4}	1.846×10^{4}
MLP	0.04	185.5420	-1.662×10^{4}	2.225×10^{4}	5.630×10^{3}
ResNet-32	0.025	41.1785	-2.528×10^{3}	5.325×10^{4}	5.072×10^{4}
ResNet-32	0.05	146.3482	-7.380×10^{3}	3.603×10^{4}	2.865×10^{4}
ResNet-32	0.1	372.7896	-1.076×10^{4}	2.451×10^{4}	1.375×10^{4}
Transformer	0.05	15.6687	1.629×10^{2}	5.100×10^{4}	5.116×10^{4}
Transformer	0.1	111.8465	-7.767×10^{2}	2.464×10^{4}	2.387×10^{4}
Transformer	0.2	405.8404	-2.682×10^{3}	1.336×10^{4}	1.067×10^{4}

TABLE V: Retrospective check of the local loss approximation for all nine conditions. The unthresholded count and median e_t use 2,999 transitions per condition. Each threshold cell gives opposite signs / eligible transitions; eligibility requires both change magnitudes to exceed $h_t(\tau)$. These descriptive temporal counts are not independent-sample predictive accuracies.

				Opposite signs / eligible transitions at threshold τ		
Model	η	No threshold	Median e_t	10^{-7}	10^{-6}	10^{-5}
MLP	0.004	0/2999	2.905×10^{-4}	0/2999	0/2999	0/2999
MLP	0.02	145/2999	6.146×10^{-4}	145/2998	145/2998	141/2989
MLP	0.04	281/2999	7.343×10^{-3}	281/2999	281/2998	280/2994
ResNet-32	0.025	595/2999	1.030×10^{-2}	595/2999	592/2991	570/2934
ResNet-32	0.05	694/2999	2.347×10^{-2}	694/2999	692/2997	682/2978
ResNet-32	0.1	814/2999	2.324×10^{-2}	813/2997	813/2997	809/2989
Transformer	0.05	0/2999	2.381×10^{-5}	0/2999	0/2999	0/2999
Transformer	0.1	542/2999	1.398×10^{-2}	541/2997	534/2977	436/2703
Transformer	0.2	1325/2999	6.323×10^{-2}	1324/2996	1319/2989	1239/2837

TABLE VI: Final spectral context at parameter state 3000. Only the MLP measurement covers its complete training objective. ResNet and Transformer use fixed subsets of the stated size; residual-checked eigenpairs do not establish subset representativeness. All conditions use seed 0.

Model	η	Hessian objective coverage	$\eta\lambda_{\max,3000}$
MLP	0.004	Full 5,000 images	0.3168
MLP	0.02	Full 5,000 images	2.2157
MLP	0.04	Full 5,000 images	2.3259
ResNet-32	0.025	Fixed 5,000-image subset	2.8176
ResNet-32	0.05	Fixed 5,000-image subset	2.6599
ResNet-32	0.1	Fixed 5,000-image subset	3.2488
Transformer	0.05	Fixed 2,500-sequence subset	1.7229
Transformer	0.1	Fixed 2,500-sequence subset	2.2752
Transformer	0.2	Fixed 2,500-sequence subset	3.1202

REFERENCES

[1] J. Hestness, S. Narang, N. Ardalani, G. Diamos, H. Jun, H. Kianinejad, M. Patwary, Y. Yang, and Y. Zhou, "Deep learning scaling is predictable, empirically," *arXiv preprint arXiv:1712.00409*, 2017.

[2] J. Kaplan, S. McCandlish, T. Henighan *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

[3] T. Henighan *et al.*, "Scaling laws for autoregressive generative modeling," *arXiv preprint arXiv:2010.14701*, 2020.

[4] J. Hoffmann, S. Borgeaud, A. Mensch, E. Cassirer, T. Jones, T. Dasilva, K. Tunyasuvunakool, C. Humphreys, E. Rutherford, E. Buchatskaya *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[5] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: Training imagenet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.

[6] L. N. Smith, "Cyclical learning rates for training neural networks," in *2017 IEEE winter conference on applications of computer vision (WACV)*. IEEE, 2017, pp. 464–472.

[7] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International conference on machine learning*. pmlr, 2015, pp. 448–456.

[9] A. Krogh and J. Hertz, "A simple weight decay can improve generalization," *Advances in neural information processing systems*, vol. 4, 1991.

[10] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, pp. 1–17, 1964.

[11] D. P. Bertsekas, "Nonlinear programming," *Journal of the Operational Research Society*, vol. 48, no. 3, pp. 334–334, 1997.

[12] S. Jastrzębski, M. Daniluk, and K. Cho, "The break-even point on optimization trajectories of deep neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[13] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, "Gradient descent on neural networks typically occurs at the edge of stability," in *International Conference on Learning Representations (ICLR)*, 2021.

[14] S. Arora, Z. Li, and A. Panigrahi, "Understanding gradient descent on the edge of stability in deep learning," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 948–1024.

[15] A. Damian, E. Nichani, and J. D. Lee, "Self-stabilization: The implicit bias of gradient descent at the edge of stability," in *International Conference on Learning Representations (ICLR)*, 2023.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[17] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," in *International Conference on Learning Representations (ICLR)*, 2021.

[18] L. Sagun, U. Evci, V. U. Guney, Y. Dauphin, and L. Bottou, "Empirical analysis of the hessian of over-parametrized neural networks," 2017.

[19] B. Ghorbani, S. Krishnan, and Y. Xiao, "An investigation into neural net optimization via hessian eigenvalue density," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 2232–2241.

[20] S. Mandt, M. D. Hoffman, and D. M. Blei, "Stochastic gradient descent as approximate bayesian inference," *Journal of Machine Learning Research*, vol. 18, pp. 1–35, 2017. [Online]. Available: https://www.jmlr.org/papers/v18/17-214.html

[21] W. Hu, C. J. Li, L. Li, and J.-G. Liu, "On the diffusion approximation of nonconvex stochastic gradient descent," *Annals of Mathematical Sciences and Applications*, vol. 4, no. 1, pp. 3–32, 2019.

[22] S. Mandt, M. D. Hoffman, and D. M. Blei, "Stochastic gradient descent as approximate bayesian inference," *Journal of Machine Learning Research*, vol. 18, no. 1, pp. 4873–4907, 2017.

[23] S. Jastrzebski, Z. Kenton, D. Arpit, N. Ballas, A. Fischer, A. Storkey, and Y. Bengio, "Three factors influencing minima in sgd," in *International Conference on Learning Representations (ICLR)*, 2018.

[24] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts *et al.*, "PaLM: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022. [Online]. Available: https://arxiv.org/abs/2204.02311v5

[25] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen *et al.*, "OPT: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022. [Online]. Available: https://arxiv.org/abs/2205.01068v4

[26] Team OLMo, P. Walsh, L. Soldaini, D. Groeneveld, K. Lo *et al.*, "2 OLMo 2 furious," *arXiv preprint arXiv:2501.00656*, 2025. [Online]. Available: https://arxiv.org/abs/2501.00656v3

[27] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: https://arxiv.org/abs/2407.21783v3

[28] J. Z. HaoChen, C. Wei, J. Lee, and T. Ma, "Shape matters: Understanding the implicit bias of the noise covariance," in *Conference on Learning Theory*. PMLR, 2021, pp. 2315–2357.

[29] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[30] S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. van der Wal, "Pythia: A suite for analyzing large language models across training and scaling," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 2397–2430.

[31] J. Kaddour, "The minipile challenge for data-efficient language models," *arXiv preprint arXiv:2304.08442*, 2023.

[32] PyTorch contributors, "PyTorch examples: Word language model," GitHub repository, revision `13acec6d7c78`; full commit specified in the URL. [Online]. Available: https://github.com/pytorch/examples/tree/13acec6d7c78dacd5e1fe9b0b4a325e1d39abc15/word_language_model